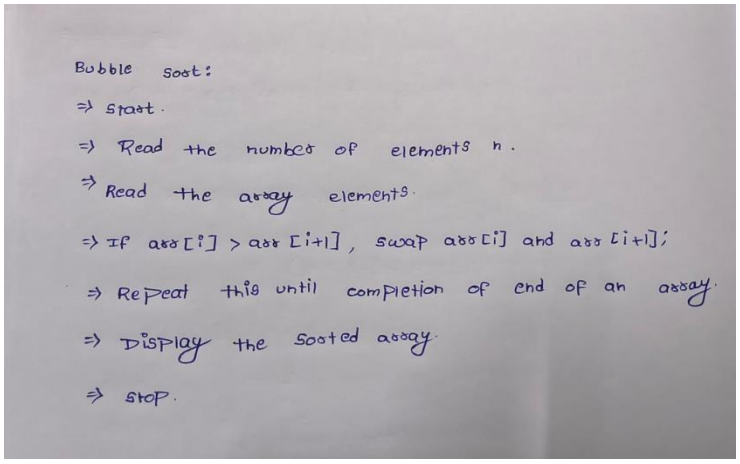
PRACTICAL-01

AIM: To implementation and time analysis of sorting algorithms.Bubble sort,Selection Sort,Insertion sort,Merge sorta and Quick sort.

1.BUBBLE SORT

ALGORITHM:



PROGRAM:

```
#include    <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
```

```
}

int main() {
int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++) {

        cin >> arr[i];
    }


    bubbleSort(arr, n);


    cout << "Sorted elements: ";
    for (int i = 0; i < n; i++) {

        cout << arr[i] << " ";
    }
    cout << endl;


    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter array elements: 7
2
9
1
5
Sorted elements: 1 2 5 7 9
```

TIME COMPLEXITY:

```
for (int i=0; i<n-1; i++) ..... outer loop
{
    for (int j=0; j<n-1; j++) ..... inner loop
    {
        if (arr[j] > arr[j+1]) .....
        {
            int temp = arr[j]; .....
            arr[j] = arr[j+1]; .....
            arr[j+1] = temp; .....
        }
    }
}
```

=> outer loop: Runs i=0 to n-1
so it executes n-1 times.

=> Inner loop : pass(i) iterations

0	n-1
1	n-2
:	:
n-2	1

$(n-1)+(n-2)+\cdots+2+1 = 1+2+3+\cdots+(n-1) = \frac{n(n-1)}{2}$

$= \frac{n^2-n}{2}$

$T(n) = n-1 + \frac{n^2-n}{2} + 1+1+1+1$

$= n + \frac{n^2-n}{2} + 3$

$= \frac{2n+n^2-n+6}{2} = \frac{n^2+n+6}{2}$ ignore all constant

$= n^2-n$ by asymptotic

$T(n) = O(n^2)$ highest power notation (∵ c, 2)

Worst case:
condition: Array is in descending order.
Process: every comparison leads to swap.
comparisons: $(n-1)+(n-2)+\cdots+1 = \frac{n(n-1)}{2}$

$\frac{n(n-1)}{2} = \frac{n^2-n}{2} = n^2-n$

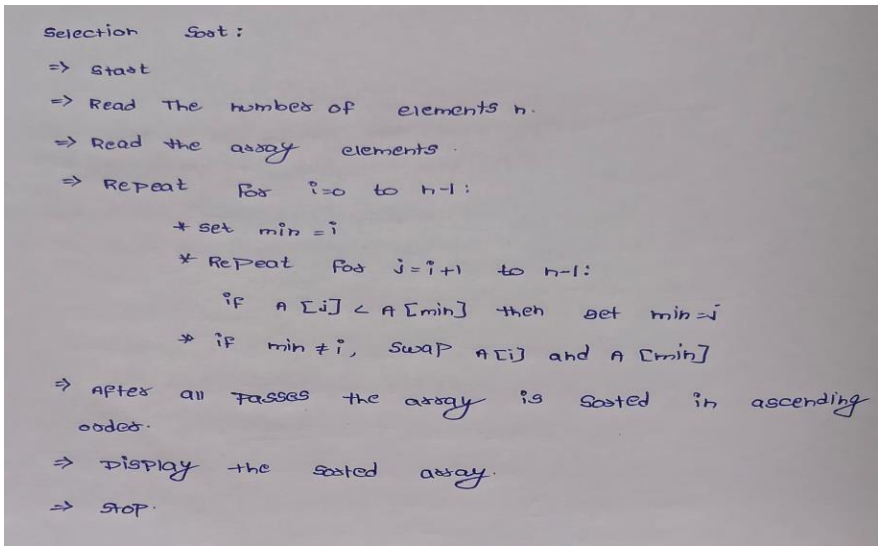
Time compl = $O(n^2)$

Average case:
condition: Array element are in random order.
Process: About half of the comparisons leads to swap.
comparisons: still $\frac{n(n-1)}{2}$ Time complexity: $O(n^2)$

Best case:
condition: Array is already sorted
Process: comparisons occurs but no swaps.
Time complexity: $O(n)$

SELECTION SORT:

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
int main() {
```

```
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

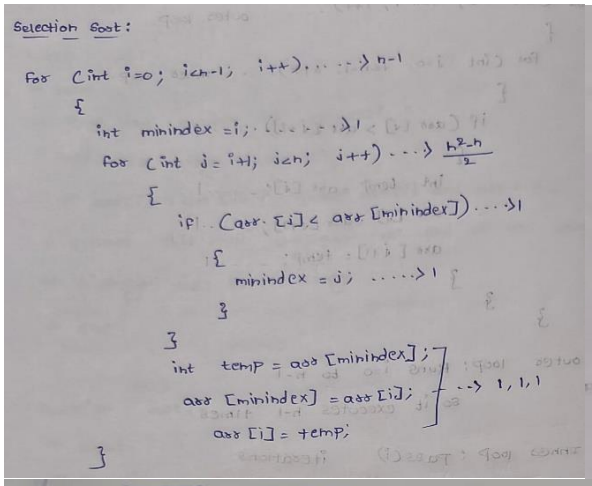
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    cout << "Original array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
        cout << endl;
    }
   selectionSort(arr, n);

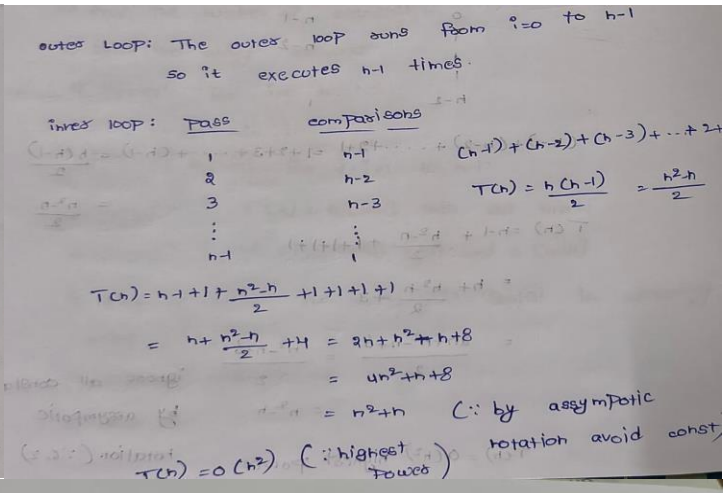
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    //cout << endl;
    }
    return 0;
}
```

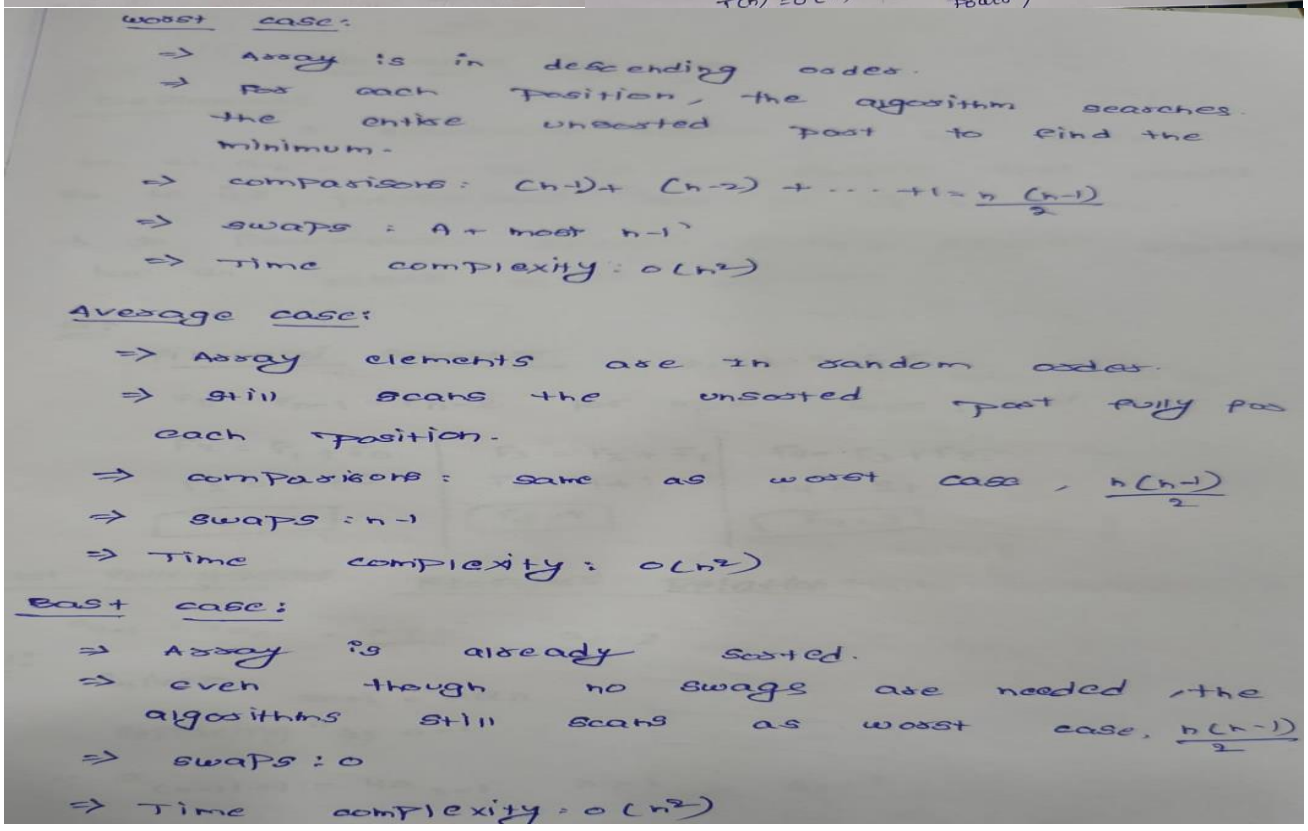
OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 9
2
4
6
0
Original array: 9
2
4
6
0
Sorted array: 0 2 4 6 9
```

TIME COMPLEXITY:

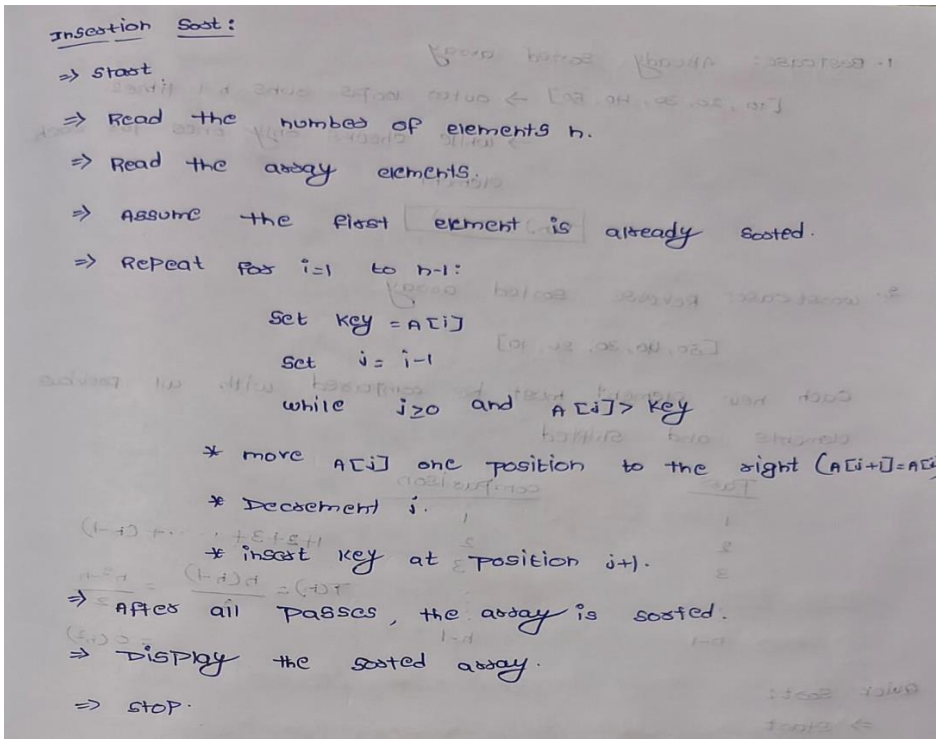






2. INSERTION SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;



        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
```

```
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

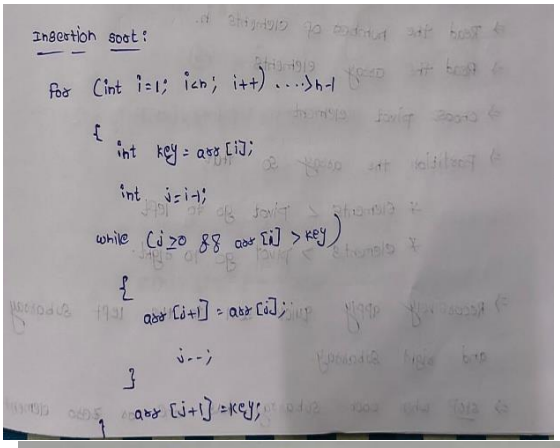
    cout << "Original array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
        cout << endl;
    }
    insertionSort(arr, n);

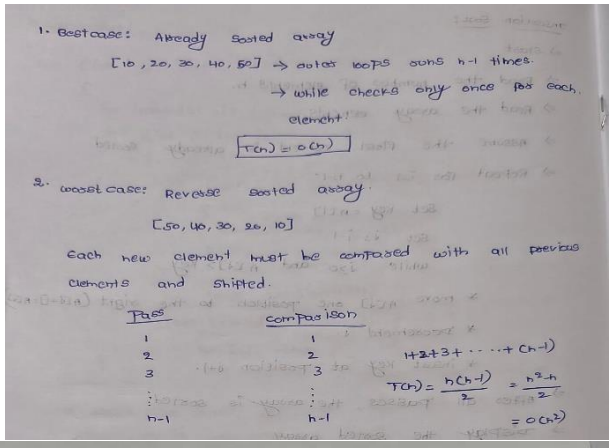
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
        //cout << endl;
    }
    return 0;
}
```

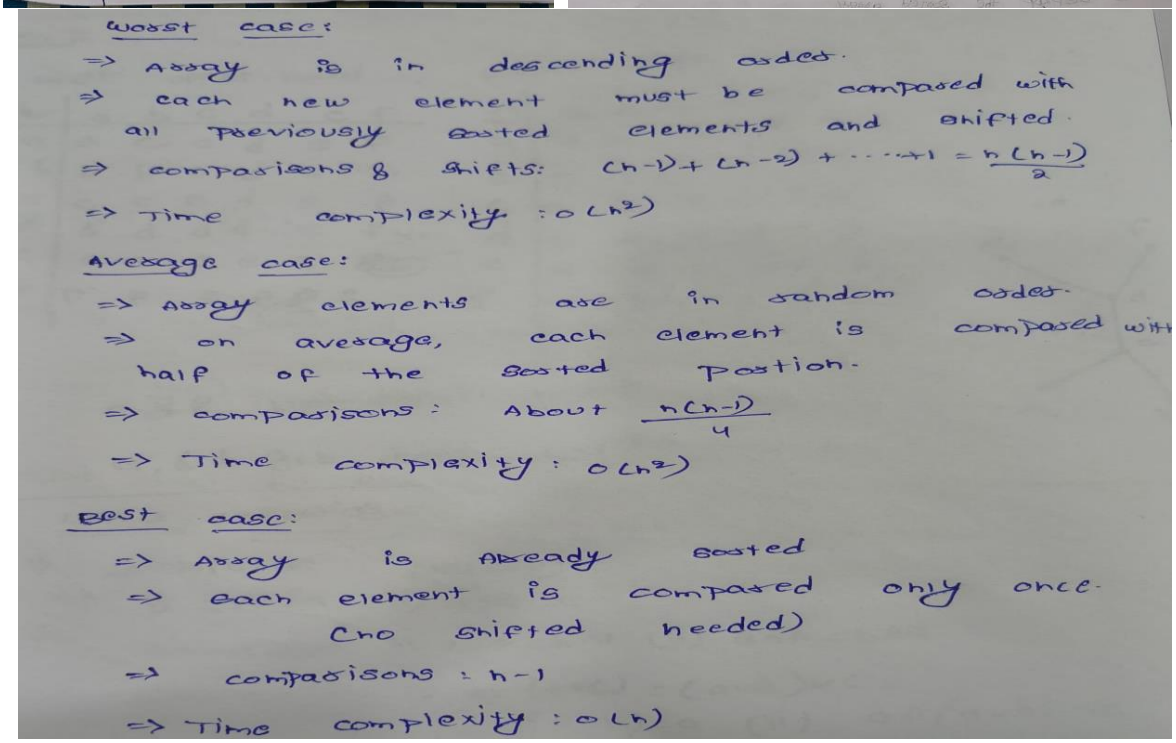
OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 7
3
8
2
0
Original array: 7
3
8
2
0
Sorted array: 0 2 3 7 8
```

TIME COMPLEXITY:

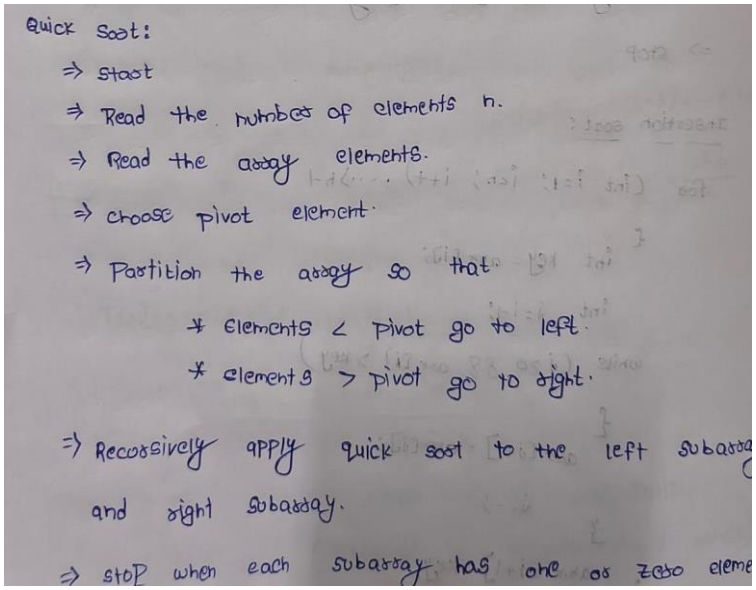






3. QUICK SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;
int partition(int arr[], int lb, int ub) { int
    pivot = arr[lb];
    int start = lb; int
    end = ub;

    while (start < end) {
        while (arr[start] <= pivot ) {
            start++;
        }
        while (arr[end] > pivot) {
            end--;
```

```
        }

        if (start < end) {
            swap(arr[start], arr[end]);
        }
    }
    swap(arr[lb], arr[end]);
    return end;
}

void quickSort(int arr[], int lb, int ub) { if
    (lb < ub) {
        int loc = partition(arr, lb, ub);
        quickSort(arr, lb, loc - 1); quickSort(arr,
        loc + 1, ub);
    }
}
int main() {
    int n;
    cout << "Enter size of array: "; cin
    >> n;

    int arr[n];
    cout << "Enter " << n << " elements: "; for
    (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
```

```
    quickSort(arr, 0, n - 1);

    cout << "Sorted array: "; for
    (int i = 0; i < n; i++) { cout
            << arr[i] << " ";
    }
    return 0;
}
```

OUTPUT:

```
Enter size of array: 5
Enter 5 elements: 7
3
8
2
0
Sorted array: 0 2 3 7 8
```

TIME COMPLEXITY:

Quick Sort:

⇒ The Pivot divides the array into two equal halves

→ $2T(n/2)$

⇒ For Sorting we use n elements at last.

$O(n)$

$T(n) = 2T(n/2) + n$

$T(n) = 2T(n/2) + n \rightarrow (1)$

$n \rightarrow n/2$ Put in (1)

$T(n/2) = 2T(n/4) + n/2$

$T(n) = 2T(n/4) + n/2 \rightarrow (2)$

sub (2) in (1)

$T(n) = 2(2T(n/4) + n/2) + n$

$= 2^2 T(n/4) + \frac{2n}{2} + n$

$= 2^2 T(n/4) + 2n \rightarrow (3)$

$n \rightarrow n/4$ Put in (3)

$T(n/4) = 2^2 T(n/2^4) + 2(n/4) \rightarrow (4)$

(4) in (3)

$T(n) = 2^2 [2^2 T(n/2^4) + \frac{2^2 n}{4}] + 2n$

$= 2^4 T(n/2^4) + \frac{2^3 n}{4} + 2n$

$= 2^4 T(n/2^4) + 4n$

$T(n) = 2^k T(n/2^k) + kn$

$T(n) = 2^k T(n/2^k) + kn$

$\frac{n}{2^k} = 1$

$n = 2^k$

Apply log both sides

$\log n = \log 2^k$

$k = \log n$

$T(n) = 2^{\log n} T(n/2^{\log n}) + \log n(n)$

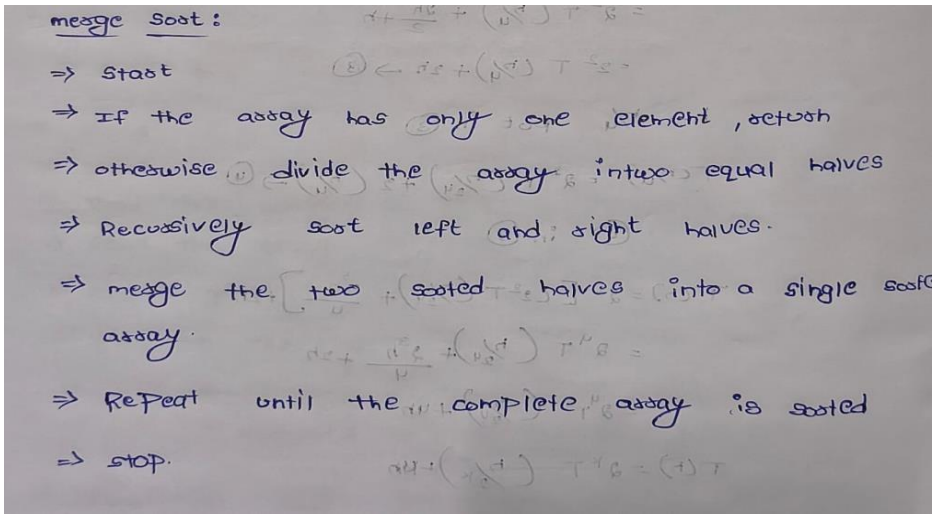
$= n^{\log 2} + \left(\frac{n}{n^{\log 2}}\right) + \log n(n)$

$= n \log n + n$

$T(n) = O(n \log n)$

4.MERGE SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;


void merge(int arr[], int left, int mid, int right) { int
    n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
     while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++]; else
        arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
int main() { int
    n;
    cout << "Enter number of elements: "; cin
    >> n;
    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

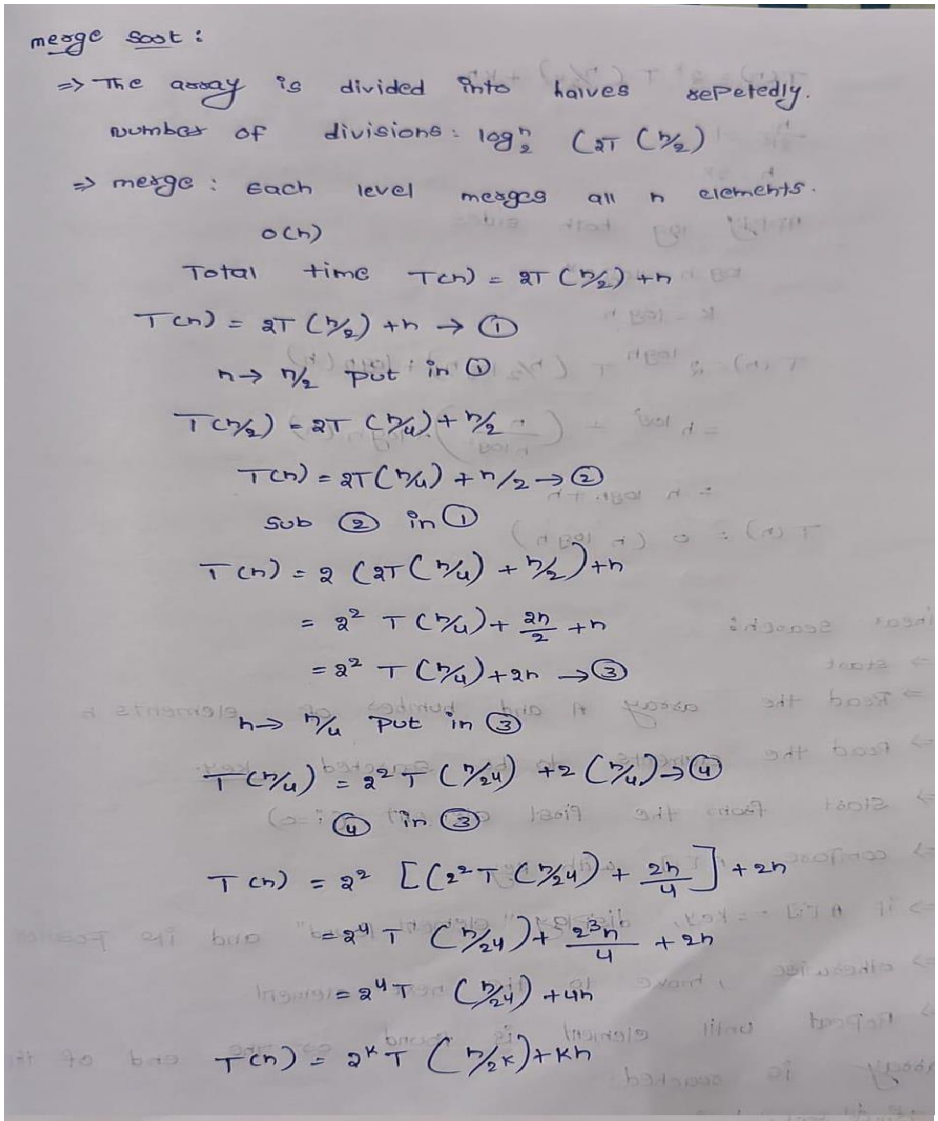
    mergeSort(arr, 0, n - 1);

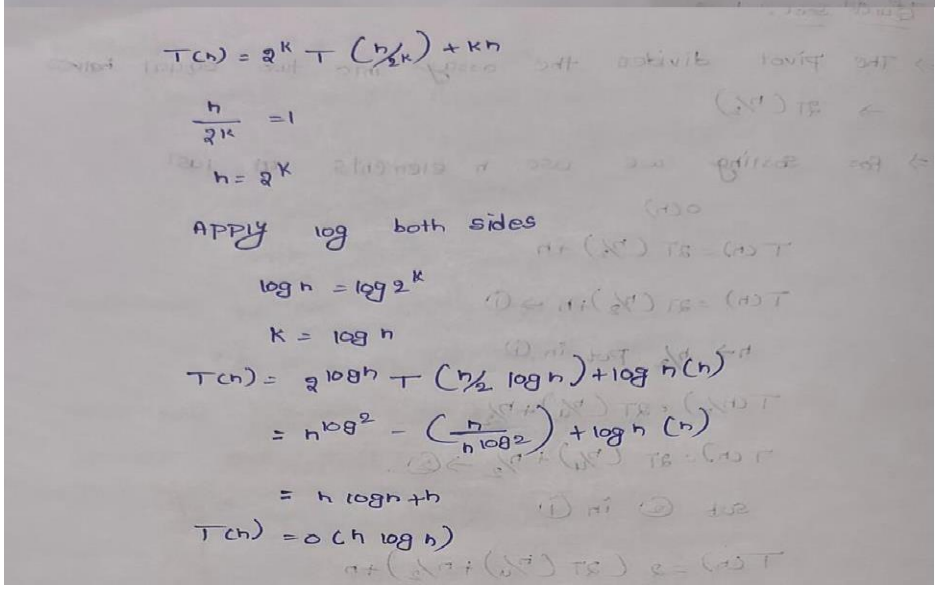
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter elements: 7
9
0
2
5
Sorted array: 0 2 5 7 9
```

TIME COMPLEXITY:





CONCLUSION:

Bubble Sort and Selection Sort are simple but inefficient with $O(n^2)$in all major cases. Insertion Sort improves to $O(n)$in the best case when the array is already sorted. Quick Sort is very efficient on average with $O(n \log n)$, though it can degrade to $O(n^2)$. Merge Sort is consistently $O(n \log n)$in all cases but requires extra space.

M.Ravi Kiran Reddy (92460118230)